Exercise 2: Black-box API Solutions

Machine Intelligence – Lecture 2

How to Use the Black-Box API

API Method	Do	Don't
`InitialState()`	Return a complete state with all variables	Return partial info or just the goal
`GoalTest(s)`	Check conditions on state s only	Modify the state or check paths
`Actions(s)`	Return only valid/applicable actions for s	Return all possible actions ignoring s
`Cost(a)`	Return consistent, non-negative costs	Make cost depend on state (use s info)
`Successor(s,a)`	Copy s first, then apply changes	Modify the original state directly

When formulating a search problem using the black-box API, follow these steps:

1. **Define the State Representation:** Choose what information fully describes a configuration of the problem. The state should contain all details needed to determine valid actions and check goal conditions.
2. **InitialState():** Return the starting configuration. This is typically given in the problem description.
3. **GoalTest(s):** Define what makes a state a solution. Translate the problem's success criteria into boolean conditions on state s.
4. **Actions(s):** List all valid moves from state s. Consider:
   - What can change in the state?
   - What constraints limit valid actions?
   - Use descriptive action names (e.g., "move-A-to-B", "toggle-X").
5. **Cost(a):** Assign a cost to each action. Use 1 for uniform cost, or problem-specific values (distance, time, etc.).
6. **Successor(s, a):** Apply action a to state s and return the new state. Always copy the state first to avoid modifying the original.

2.1 Sketch pseudo-code of the API for problem formulations

Based on the black-box API definition:

- **InitialState()**: Returns the initial state
- **GoalTest(s)**: Returns true iff s is a goal state
- **Cost(a)**: Returns the cost of action a
- **Actions(s)**: Returns applicable actions in state s
- **Successor(s, a)**: Returns resulting state after applying a to s

Exercise 1.2: Team Assignment Problem

State: Binary vector (A, B, C, D, E, F) where 0 = Team 1, 1 = Team 2.

```
function InitialState
    return (A = 0, B = 0, C = 0, D = 0, E = 0, F = 0)        ▷ Any starting assignment works
end function
```

```
function GoalTest(s)
    T_1 ← engineers with value 0;    T_2 ← engineers with value 1
    if |T_1| ≠ 3 or |T_2| ≠ 3 then return false
    end if
    for each skill ∈ {Front-end, Back-end, DevOps} do
        if |SkillSum(T_1, skill) − SkillSum(T_2, skill)| > 2 then return false
        end if
    end for
    return true
end function
```

```
function Actions(s)
    acts ← ∅
    for each engineer X ∈ {A, B, C, D, E, F} do
        if s[X] = 0 then acts ← acts ∪ {switch-X-to-Team2}
        else acts ← acts ∪ {switch-X-to-Team1}
        end if
    end for
    return acts
end function
```

```
function Cost(a)
    return 1                                                ▷ Uniform cost
end function
```

```
function Successor(s, a)
    s' ← copy(s)
    Parse a as "switch-X-to-Teamv"
    s'[X] ← v                                               ▷ v ∈ {1, 2} maps to {0, 1}
    return s'
end function
```

Exercise 1.3: Meeting Point Problem

State: Triplet $(alice, emil, frank)$ – locations of Alice, Emil, Frank.

```
function InitialState
    return (alice = A, emil = E, frank = F)
end function
function GoalTest(s)
    return s.alice = s.emil = s.frank
end function
function Actions(s)
    acts ← ∅
    for each person p ∈ {alice, emil, frank} do
        for each neighbor n of s[p] in the city map do
            acts ← acts ∪ {move-p-from-s[p]-to-n}
        end for
    end for
    return acts
end function
function Cost(a)
    Parse a as "move-p-from-ℓ1-to-ℓ2"
    return distance(ℓ1, ℓ2)                    ▷ Edge weight from city map
end function
function Successor(s, a)
    s′ ← copy(s)
    Parse a as "move-p-from-ℓ1-to-ℓ2"
    s′[p] ← ℓ2
    return s′
end function
```

Exercise 1.4: Elevator Scheduling Problem

State: $(elev, p_1, p_2, p_3)$ where $elev$ = elevator floor, p_i = passenger i location (floor number or E for inside elevator).

```
function InitialState
    return (elev = 1, p1 = 1, p2 = 1, p3 = 3)
end function
function GoalTest(s)
    return s.p1 = 3 and s.p2 = 2 and s.p3 = 2
end function
function Actions(s)
    acts ← ∅
    if s.elev < F then acts ← acts ∪ {up}
    end if
    if s.elev > 1 then acts ← acts ∪ {down}
    end if
    occupancy ← |{i : s.pi = E}|                ▷ Count passengers in elevator
    for each passenger i do
```

```
        if s.p_i = s.elev and occupancy < C then acts ← acts ∪ {board-i}
        end if
        if s.p_i = E then acts ← acts ∪ {exit-i}
        end if
    end for
    return acts
end function
```

```
function COST(a)
    if a ∈ {up, down} then return 1
    else return 0                                   ▷ board/exit are free
    end if
end function
```

```
function SUCCESSOR(s, a)
    s' ← copy(s)
    if a = up then s'.elev ← s.elev + 1
    else if a = down then s'.elev ← s.elev − 1
    else if a = board-i then s'.p_i ← E             ▷ Passenger enters elevator
    else if a = exit-i then s'.p_i ← s.elev         ▷ Passenger exits at current floor
    end if
    return s'
end function
```